# Active Fault-Tolerant Control of a Quadrotor with a Total Rotor Failure: Re-implementation on X500 Parameters

**Abstract**

We re-implement the active fault-tolerant control (AFTC) scheme of Lu and van Kampen (IROS 2015) in Python, using the parameters of the PX4 Gazebo X500. The failed rotor is detected 74 ms after the fault and the drone returns to hover, but only if the motors can deliver about 1400–1500 rad/s instead of the stock 1000 rad/s. The recovery also depends strongly on the yaw drag $k_r$, which is not given by the X500 model: at the value used here (spin of about 31 rad/s) recovery takes 3.5 s, while for $k_r \geq 0.1$ it fails.

## 1 Introduction

This report summarises a Python re-implementation of the paper *"Active fault-tolerant control for quadrotors subjected to a complete rotor failure"* by Lu and van Kampen [1]. The paper builds a complete system for the case where one rotor stops working completely. It has two parts:

- **FDI** (fault detection and isolation): it finds out that a rotor has failed and which one.
- **FTC** (fault-tolerant control): it keeps the quadrotor flying using only the three remaining rotors.

We reproduce this with the parameters of the PX4 Gazebo X500 drone, but in a pure Python simulation (no Gazebo yet). Rotors are numbered 0–3 as in the X500 SDF.

## 2 How the Method Works

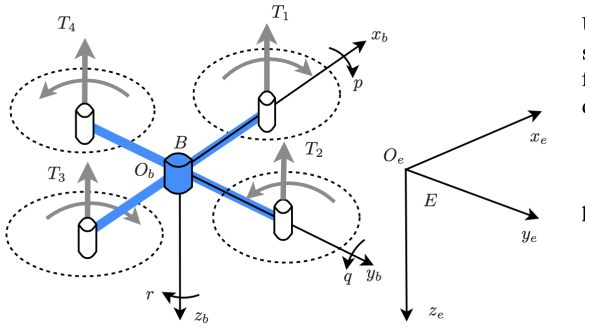



Figure 1: Body and earth frames used in the model (reproduced from Lu and van Kampen, 2015). The $z$ axes point down.

### 2.1 Detecting the failed rotor

Each rotor $i$ has a loss-of-effectiveness (LOE) factor $l_i$: $l_i = 0$ means healthy and $l_i = 1$ means completely dead. The total thrust $T$ and the three torques $\tau_x, \tau_y, \tau_z$ are estimated from the measured motion (filtered gyro derivatives and vertical acceleration). They are then compared with what the motor commands should have produced:

$$\hat{\mathbf{l}} = \mathbf{1} - \mathbf{A}^{-1} \begin{bmatrix} \hat{T} & \hat{\tau}_x & \hat{\tau}_y & \hat{\tau}_z \end{bmatrix}^T, \qquad \mathbf{A} = \mathbf{A}_h \,\mathrm{diag}(\hat{\omega}_0^2, \ldots, \hat{\omega}_3^2), \tag{1}$$

where $\hat{\omega}_i$ comes from the motor commands through the actuator model, and $\mathbf{A}_h$ is given in Eq. (2) (with $\eta_i = 1$). If $\hat{l}_i \geq l_T$ for $N$ consecutive samples, rotor $i$ is declared failed, and from then on $l_i = 1$ for that rotor and 0 for the others (paper Eq. (41)).

### 2.2 Flying with three rotors

Three rotors cannot control roll, pitch, yaw and thrust together, so the paper gives up yaw control. The quadrotor spins about its vertical axis and only the direction of the thrust axis is controlled:

- An outer loop turns the desired position into a desired thrust direction.
- An inner loop (INDI, incremental nonlinear dynamic inversion) tracks the angular rates $p, q$ using the estimated (filtered) angular acceleration.
- A reduced control allocation sends thrust and the two torques to the three working rotors.

### 2.3 Equations used in this implementation

**Mixing matrix for the X layout.** Take $x$ forward, $y$ right, $z$ down, with rotor $i$ at $(x_i, y_i)$ pushing with force $-T_i \mathbf{e}_z$. Its torque is $\mathbf{r}_i \times \mathbf{F}_i = (-y_i T_i,\ x_i T_i,\ 0)$, so $\tau_x = -\sum y_i T_i$ and $\tau_y = \sum x_i T_i$. The yaw torque of each rotor is $\sigma_i k_m T_i$, with $\sigma_i = +1$ for the ccw rotors (0, 1) and $-1$ for the cw rotors (2, 3). With $T_i = b\,\omega_i^2$, $d = k_m b$ and $l = 0.174$ m, the rotor positions of the SDF give

$$\begin{bmatrix} T \\ \tau_x \\ \tau_y \\ \tau_z \end{bmatrix} = \underbrace{\begin{bmatrix} b & b & b & b \\ -lb & lb & lb & -lb \\ lb & -lb & lb & -lb \\ d & d & -d & -d \end{bmatrix}}_{\mathbf{A}_h} \begin{bmatrix} \omega_0^2 \\ \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \end{bmatrix}. \tag{2}$$

This replaces the paper's Eq. (20), which is for a plus layout.

**Three-rotor allocation.** After rotor $k$ fails, its column and the yaw row are removed from $\mathbf{A}_h$. The remaining $3 \times 3$ matrix is invertible as long as the three remaining rotors are not collinear, and

$$[\omega_i^2]_{i \neq k} = \mathbf{A}_{h,3}^{-1} \begin{bmatrix} T_c & \tau_{xc} & \tau_{yc} \end{bmatrix}^T. \tag{3}$$

**Primary axis and INDI.** Let $\hat{\mathbf{u}}$ be the desired thrust direction (earth frame) and $\mathbf{n} = \mathbf{R}^T \hat{\mathbf{u}}$ the same vector in body axes. Since $\hat{\mathbf{u}}$ is fixed in space, $\dot{\mathbf{n}} = -\boldsymbol{\omega} \times \mathbf{n}$, and its first two components are

$$\begin{bmatrix} \dot{n}_x \\ \dot{n}_y \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & -n_z \\ n_z & 0 \end{bmatrix}}_{\mathbf{B}_2} \begin{bmatrix} p \\ q \end{bmatrix} + \begin{bmatrix} n_y \\ -n_x \end{bmatrix} r. \tag{4}$$

$\mathbf{B}_2$ has determinant $n_z^2$, so the desired rates are $[p_d, q_d]^T = \mathbf{B}_2^{-1}\big(\boldsymbol{\nu} - [n_y, -n_x]^T r\big)$ with $\boldsymbol{\nu} = -K_n[n_x, n_y]^T$, which drives $\mathbf{n}$ to the primary axis $[0, 0, 1]^T$. The inner loop uses the incremental form

$$\tau_c = \tau_0 + I\,(v - \dot{\omega}_0), \tag{5}$$

where $\tau_0 = \mathbf{A}_h\,\hat{\omega}^2$ (filtered) is the current torque and $\dot{\omega}_0$ comes from the second-order filter $\omega_n^2 s/(s^2 + 2\xi\omega_n s + \omega_n^2)$ with $\xi = 0.8$, $\omega_n = 20$ rad/s. The rotor model is passed through the same filter as the gyro, so both have the same delay.

**Equilibrium of the spinning vehicle.** With $p = q = 0$ and constant $r$, the roll and pitch equations give $\tau_x = \tau_y = 0$, and the vertical equation gives $T = mg$. For rotor 0 failed, using the positions of rotors 1, 2, 3, $\tau_x = l(T_1 + T_2 - T_3) = 0$ and $\tau_y = l(-T_1 + T_2 - T_3) = 0$. Adding $T_1 + T_2 + T_3 = mg$ gives

$$T_1 = 0, \qquad T_2 = T_3 = \tfrac{mg}{2}, \qquad \omega_{2,3} = \sqrt{\tfrac{mg}{2b}} \approx 1071 \text{ rad/s}. \tag{6}$$

Both live rotors are cw, so $\tau_z = -k_m\,mg$ and the yaw equation $I_z\dot{r} = \tau_z - k_r r$ gives

$$r_e = \frac{\tau_z}{k_r} = -\frac{k_m\,mg}{k_r} = -31.4 \text{ rad/s for } k_r = 0.01. \tag{7}$$

# 3 Implementation Setup

All dynamics follow the paper's model (Section II), with friction forces included and hub forces left out. The parameters come from the X500 SDF:

| Quantity | Paper | This work (X500) |
|---|---|---|
| Mass | not stated | 2.0 kg |
| Inertia $I_x = I_y$ | $6.2 \times 10^{-3}$ kg m$^2$ | 0.02167 kg m$^2$ |
| Inertia $I_z$ | $1.2 \times 10^{-2}$ kg m$^2$ | 0.04 kg m$^2$ |
| Rotor inertia $I_r$ | $6 \times 10^{-5}$ kg m$^2$ | $2.65 \times 10^{-5}$ kg m$^2$ |
| Arm length | 0.232 m (plus layout) | 0.174 m per axis (X layout) |
| Thrust coefficient $b$ | $3.13 \times 10^{-5}$ | $8.55 \times 10^{-6}$ |
| Torque / thrust ratio $d/b$ | $\approx 0.024$ | 0.016 |
| Motor time constant | 0.02 s | 0.0125 s up, 0.025 s down |
| Max rotor speed | not stated | 1000 rad/s (stock), 1500 rad/s used |
| Yaw drag $k_r$ | $6 \times 10^{-3}$ | $10^{-2}$ (placeholder, not identified) |
| Fault | rotor 2 at $t = 30$ s | rotor 0 at $t = 10$ s |

Table 1: Main parameters.

The simulation uses RK4 integration at 2 ms. Gyro noise is taken from the SDF IMU noise ($8.7 \times 10^{-4}$ rad/s), and vertical velocity has 0.01 m/s noise. The fault is injected by forcing one rotor to zero speed. Unless stated otherwise the motor limit is 1500 rad/s, $k_r = 0.01$, $l_T = 0.5$ and $N = 10$. Two cases are run:

- **Hover case**: the drone hovers at 2 m altitude (comparable to the paper's first case).
- **Tracking case**: the drone follows a smooth 3D path (comparable to the paper's second case).

**Metrics.** The tracking error is $\mathbf{e} = \mathbf{p} - \mathbf{p}_{ref}$. "Recovery time" is the time after the fault until $\|\mathbf{e}\| < 5$ cm for the rest of the run. "Altitude drop" is the largest $z - z_{ref}$ after the fault, where $z$ points down. "Steady RMS" is the RMS of $\|\mathbf{e}\|$ for $t > t_{fault} + 10$ s.

# 4 Comparison with the Paper

| Item | Paper | This implementation |
|---|---|---|
| Platform | Numerical simulation, no hardware | Python simulation, same style |
| Frame layout | Plus configuration | X configuration, new allocation matrix |
| Thrust model | Blade-element coefficients with factor $\eta_i$ | Plain $k_f\omega^2$, so $\eta_i = 1$ |
| Hub forces | Included in the plant | Left out |
| Friction forces | Included in the plant | Included in the plant |
| Fault detection | Estimated $\hat{l}_i$ against threshold | Same, plus 10 consecutive samples above threshold (per rotor) |
| Delay matching | Not discussed | Motor-speed model passed through the same filter as the gyro derivative |
| Thrust estimate | Needs $\hat{\phi}, \hat{\theta}, \dot{w}$ from an optic track system | Attitude and noisy vertical velocity used directly |
| Inner loop | $\tau_c = I(v - \dot{\omega}_0)$ | $\tau_c = \tau_0 + I(v - \dot{\omega}_0)$ (full INDI form) |
| Nominal controller | Omitted from the paper | Own simple cascaded INDI controller |
| Yaw drag $k_r$ | $6 \times 10^{-3}$ | $10^{-2}$ (placeholder, not identified) |

Table 2: Main differences.

# 5 Results

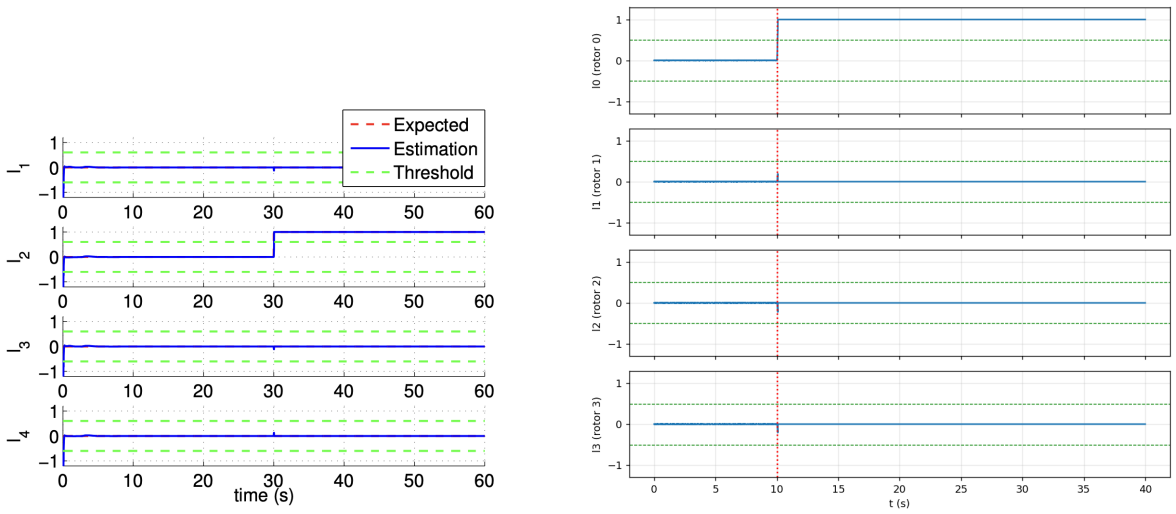
## 5.1 Hover case

The fault occurs at $t = 10$ s. The failed rotor is identified correctly and the drone returns to hover. Table 3 compares the numbers with the paper.

| Metric | Paper | This work (1500 rad/s) |
|---|---|---|
| Detection delay | about 30 ms | 74 ms |
| Back to hover | about 5 s | $\|\mathbf{e}\| < 5$ cm after 3.5 s |
| Peak error after the fault | not stated | 1.30 m (x 1.29, y 0.20, z 0.39) |
| Steady RMS error | not stated | 3.2 mm |
| Yaw after the fault | not controlled | not controlled, $-31$ rad/s |

Table 3: Hover case compared with the paper. The recovery criteria differ, so the times are indicative only.

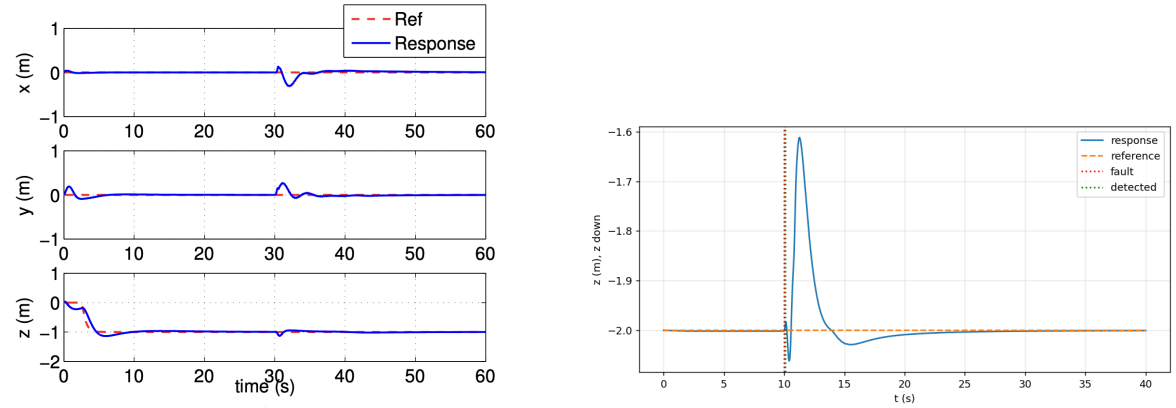
The largest error after the fault is horizontal ($x$, 1.3 m), not vertical (0.39 m).




(a) Paper (Fig. 2)

(b) This work (after detection $\hat{l}_0 = 1$, others 0)

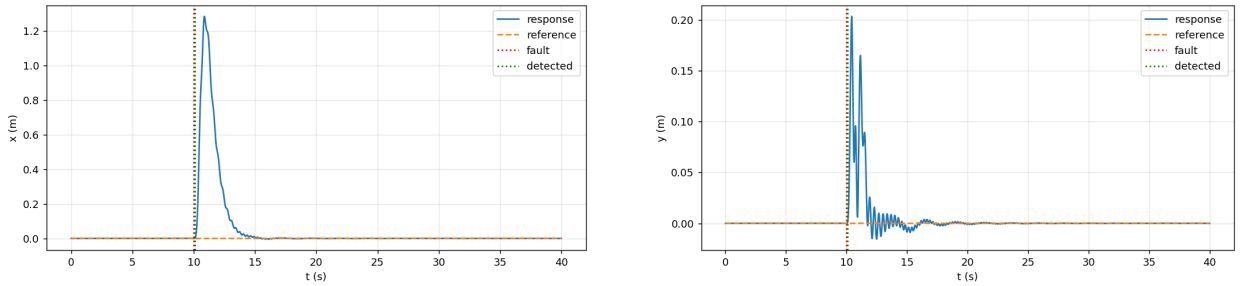
Figure 2: Estimated LOE factors. In both cases only the failed rotor rises above the threshold (dashed lines). At the moment of detection the raw estimates are $\hat{l}_0 = 0.74$ and at most 0.19 for the other rotors.




(a) Paper (Fig. 3)

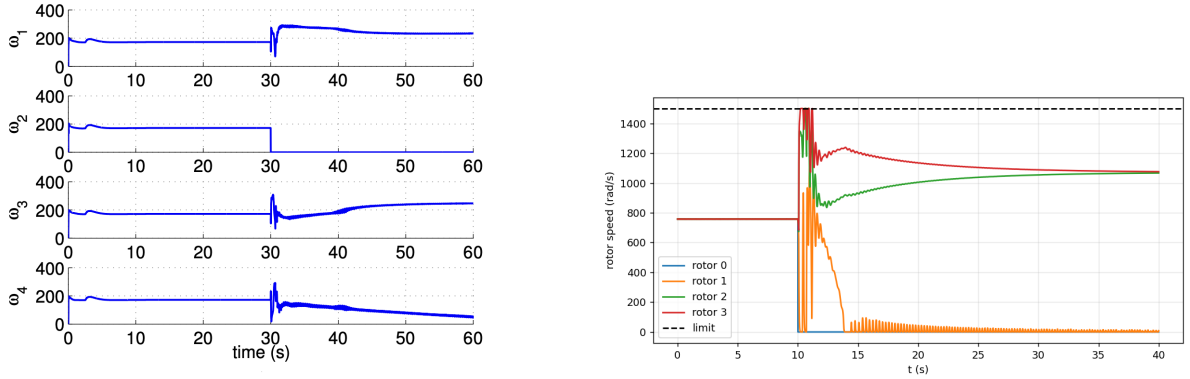
(b) This work ($z$ points down)

Figure 3: Position response in the hover case. The altitude drops by 0.39 m after the fault ($z$ moves towards zero because $z$ points down) and returns to the reference.




(a) $x$ position (b) $y$ position

Figure 4: Horizontal position in the hover case.




(a) Paper (Fig. 4) (b) This work

Figure 5: Rotor speeds. The failed rotor goes to zero. Rotor 1, diagonally opposite, decays to about zero, and rotors 2 and 3 settle near 1070 rad/s, as predicted by the equilibrium analysis. They briefly reach the 1500 rad/s limit.

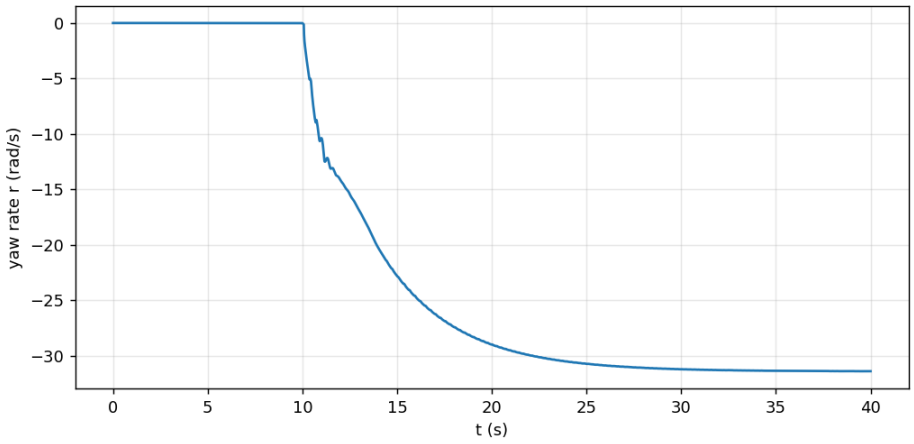
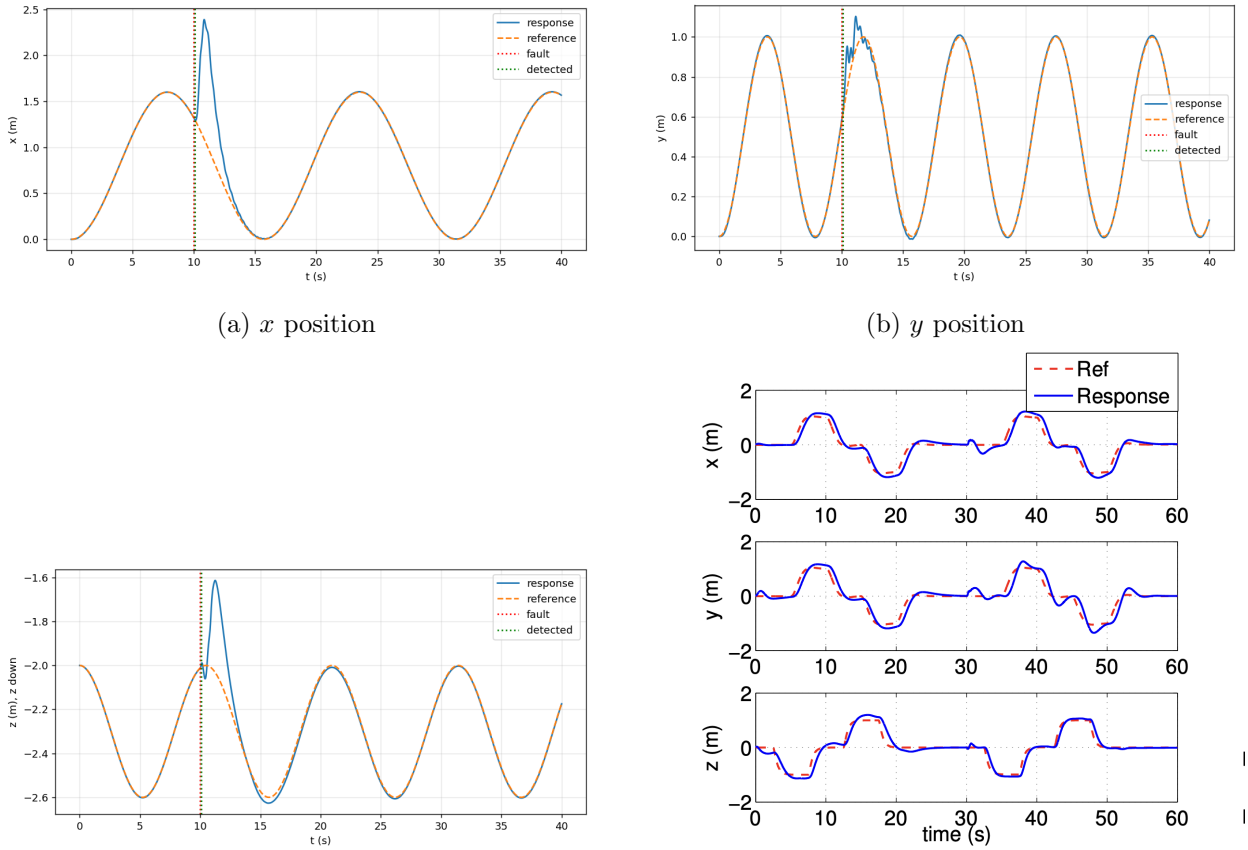



Figure 6: Yaw rate after the failure in the hover case. Yaw is not controlled, so the vehicle spins. The final value agrees with Eq. (7), and the rise time is $I_z/k_r = 4$ s.

### 5.2 Tracking case

The drone follows a smooth 3D path and the fault occurs at $t = 10$ s. The reference is $x = 0.8(1 - \cos 0.4t)$, $y = 0.5(1 - \cos 0.8t)$, $z = -2 - 0.3(1 - \cos 0.6t)$, so the horizontal path is a single arc traced back and forth. After detection the drone keeps tracking. The peak error is 1.32 m (x 1.30, y 0.22, z 0.42), $\|\mathbf{e}\| < 5$ cm after 3.5 s, and the steady RMS error is 9.5 mm.




(a) $x$ position

(b) $y$ position

(c) $z$ position ($z$ down)

(d) Paper (Fig. 5)

Figure 7: Position tracking before and after the fault. The fault and detection times are marked.

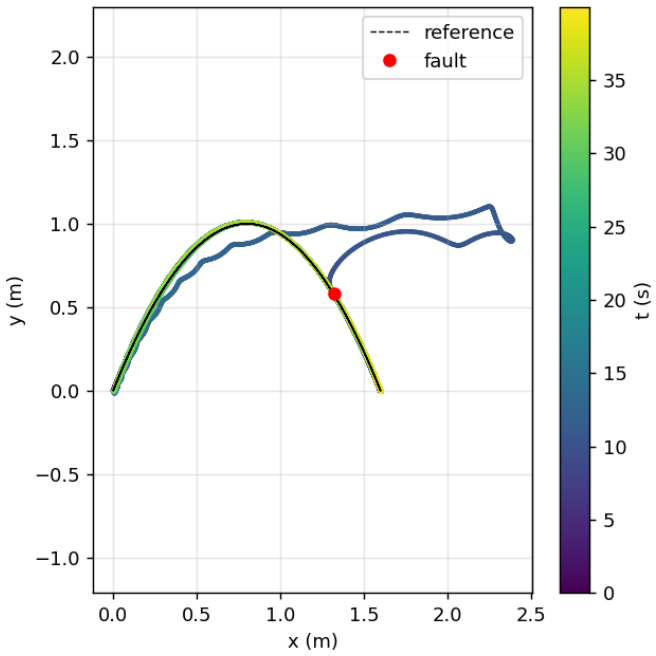


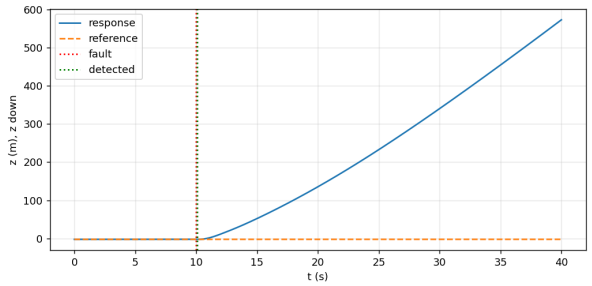

Figure 8: Horizontal path in the tracking case, coloured by time. The response leaves the reference arc after the fault (red dot) and returns to it; afterwards it lies on top of the reference.

## 6 Thrust Limit Problem

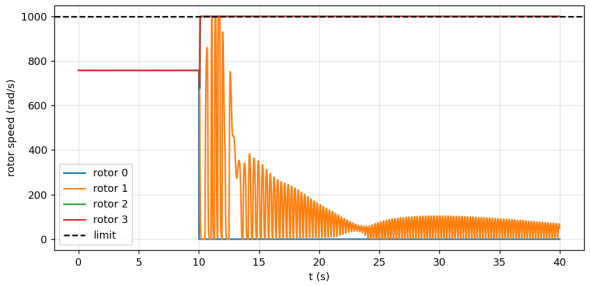
The equilibrium derived in Section 2.3 keeps the roll and pitch torques at zero. With rotor 0 dead it gives $T_1 = 0$ and $T_2 = T_3 = mg/2 = 9.81$ N, so the two diagonal rotors carry the whole weight. The stock X500 limit of 1000 rad/s gives only $k_f \omega_{max}^2 = 8.55$ N per rotor, so the motors saturate and the drone loses altitude. The minimum speed for static hover is $\omega \geq \sqrt{9.81/8.54858 \times 10^{-6}} \approx 1071$ rad/s, and the transient right after the fault needs more.

| Max rotor speed (rad/s) | Altitude drop | $\|\mathbf{e}\| < 5$ cm after | Result |
|---|---|---|---|
| 1000 (stock) | 575 m (at 40 s) | – | falls, does not recover |
| 1100 | 69 m | – | not recovered by 40 s |
| 1200 | 15 m | 9.2 s | would hit the ground |
| 1300 | 5.0 m | 5.8 s | |
| 1400 | 0.88 m | 3.9 s | |
| 1500 | 0.39 m | 3.5 s | |

Table 4: Effect of the motor speed limit (hover case, $m = 2$ kg, $k_r = 0.01$). The change is gradual, not a sharp threshold.




(a) $z$ position ($z$ down)




(b) Rotor speeds

Figure 9: Stock limit of 1000 rad/s. The live rotors stay at the limit and the altitude is lost.

A lighter vehicle needs less thrust. With the stock limit of 1000 rad/s:

| Mass (kg) | Altitude drop | $\|\mathbf{e}\| < 5$ cm after | Final yaw rate (rad/s) |
|---|---|---|---|
| 1.0 | 0.32 m | 5.8 s | $-15.7$ |
| 1.2 | 1.2 m | 4.2 s | $-18.8$ |
| 1.4 | 16 m | 10.1 s | $-21.9$ |
| 1.6 | 51 m | 23.1 s | $-23.4$ |
| 1.74 | falls | – | $-27.3$ |

Table 5: Effect of the mass at the stock limit of 1000 rad/s (hover case, $k_r = 0.01$). Note that 1.74 kg is the heaviest mass for which the equilibrium itself is possible.

The 1500 rad/s case corresponds to 19.2 N of thrust per rotor, so it should be read as a hypothetical motor upgrade, not as the stock X500. The paper does not state a motor limit. Its Fig. 4 shows rotor speeds about 25% above hover, so the limit was probably not reached there.

## 7 Sensitivity to the Yaw Drag

The yaw drag $k_r$ is not in the SDF, and it sets the spin rate through Eq. (7). For the paper's own values ($k_r = 6 \times 10^{-3}$, $d/b = 0.024$, and a mass of about 0.5 kg inferred from the 200 rad/s hover speed in its Fig. 4) the same formula gives about 20 rad/s, so a spin of 31 rad/s is not unusual. Table 6 shows that the result depends on it strongly.

| $k_r$ | predicted $\lvert r_e \rvert$ | measured $r$ | peak $\|\mathbf{e}\|$ | altitude drop | $\|\mathbf{e}\| < 5$ cm after |
|---|---|---|---|---|---|
| 0.01 | 31.4 | $-31.4$ | 1.30 m | 0.39 m | 3.5 s |
| 0.05 | 6.3 | $-6.3$ | 2.25 m | 1.17 m | 7.5 s |
| 0.1 | 3.1 | $-5.9$ | 11.4 m | 10.7 m | not recovered |
| 0.2 | 1.6 | $-2.4$ | crashes | 1650 m | – |

Table 6: Hover case, 1500 rad/s, $m = 2$ kg. Rad/s for the yaw columns. For $k_r = 0.1$ the response settles into a persistent oscillation of about 3 m (Fig. 10).

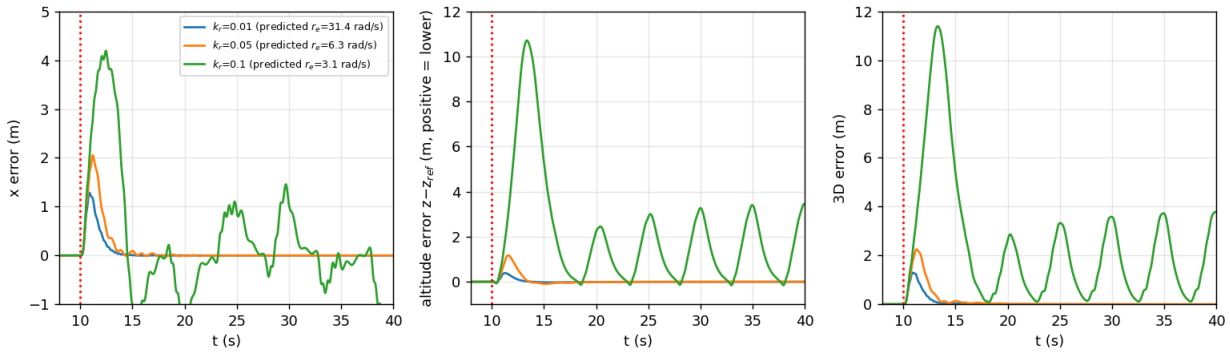



Figure 10: Error after the fault for three values of $k_r$. The 3D error for $k_r = 0.1$ does not return to zero.

Between 10.2 s and 20 s, the diagonally opposite rotor is at about zero thrust for 39%, 57% and 44% of the time for $k_r = 0.01$, 0.05 and 0.1, and the two live rotors are at the speed limit for 5%, 11% and 74% of the time. A possible explanation is that the equilibrium lies on the boundary $T_1 = 0$, so torque demands that would need negative thrust on that rotor cannot be met, and a fast spin averages these out while a slow spin does not. We have not tested this directly.

## 8 Detection Delay

The 74 ms delay is much longer than the paper's 30 ms. Only part of it comes from the 10-sample confirmation window; the rest is the lag of the second-order filter. The threshold can also be lowered: the estimates stay below 0.01 in fault-free runs, but the raw estimates of the healthy rotors reach 0.19 at detection, so $l_T$ should stay above about 0.2.

| $l_T$ | $N$ | Detection delay | Altitude drop |
|---|---|---|---|
| 0.5 | 10 | 74 ms | 0.39 m |
| 0.5 | 5 | 64 ms | 0.23 m |
| 0.5 | 2 | 58 ms | 0.15 m |
| 0.3 | 10 | 58 ms | 0.15 m |
| 0.3 | 2 | 42 ms | 0.06 m |

Table 7: Effect of the threshold and the confirmation window (hover case, 1500 rad/s). No false alarm occurred in a fault-free tracking run with $l_T = 0.25$.

## 9 Limitations and Next Steps

- The yaw drag $k_r$ is a guess and sets the spin rate; the recovery depends on it (Table 6). It should be identified from the simulator.
- Hub forces and the blade-element thrust model are not included.
- The threshold $l_T$ was fixed at 0.5 for the main results. A lower value detects faster, as shown above, but it was not tuned systematically.

- The detection delay (74 ms) is longer than the paper's, and the drone moves about 1.3 m horizontally and 0.4 m vertically before the fault-tolerant controller takes over.
- The scheme needs a motor limit well above the stock 1000 rad/s, or a vehicle of about 1.2 kg or less.
- The next step is to port the controller to Gazebo with the X500 model, raise the motor limit (or lower the mass) and identify the yaw drag.

## 10 Conclusion

The main idea of the paper works on X500 parameters: the failed rotor is detected quickly, and the quadrotor returns to hover and keeps tracking its reference using three rotors. The stock motor speed limit of 1000 rad/s is too low: recovery needs about 1400–1500 rad/s at 2 kg, or a mass of about 1.2 kg at the stock limit. The result also depends on the yaw drag, which is not part of the X500 model: with $k_r = 0.01$ the recovery takes 3.5 s, but with $k_r \geq 0.1$ it fails.

## References


[1] P. Lu and E.-J. van Kampen, "Active fault-tolerant control for quadrotors subjected to a complete rotor failure," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS)*, Hamburg, 2015, pp. 4698–4703.

[2] A. Lanzon, A. Freddi, and S. Longhi, "Flight control of a quadrotor vehicle subsequent to a rotor failure," *J. Guidance, Control, and Dynamics*, vol. 37, no. 2, pp. 580–591, 2014.

[3] M. W. Mueller and R. D'Andrea, "Stability and control of a quadrocopter despite the complete loss of one, two, or three propellers," in *IEEE Int. Conf. on Robotics and Automation (ICRA)*, 2014, pp. 45–52.

[4] PX4 Autopilot, "PX4-gazebo-models: x500 model," `model.sdf` and `x500_base/model.sdf`.